

Bachelorarbeit

**Entwicklung eines
framework-agnostisches Verfahrens zur
Optimierung von Testläufen**

Ruben Leander Rosenmeyer

Gutachter: Prof. Dr. Peter Thiemann

Betreuer: Dr. Michael Sperber

Albert-Ludwigs-Universität Freiburg

Technische Fakultät

Institut für Informatik

13. Juli 2026

Bearbeitungszeit

13. 04. 2026 – 13. 07. 2026

Gutachter

Prof. Dr. Peter Thiemann

Betreuer

Dr. Michael Sperber

ERKLÄRUNG

Hiermit erkläre ich, dass ich diese Abschlussarbeit selbständig verfasst habe, keine anderen als die angegebenen Quellen/Hilfsmittel verwendet habe und alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten Schriften entnommen wurden, als solche kenntlich gemacht habe.

Darüber hinaus erkläre ich, dass diese Abschlussarbeit nicht, auch nicht auszugsweise, bereits für eine andere Prüfung angefertigt wurde.

Ort, Datum

Unterschrift

Hilfsmittel

Bei der Erstellung dieser Bachelorarbeit und der begleitenden Inhalte wurden folgende Tools als Hilfsmittel eingesetzt:

- ChatGPT von OpenAI, Modell GPT-5.5
zur Fehlerbehebung und allgemeiner Unterstützung des Schreib- und Codeerstellungsprozesses, sowie Erstellung von tikz-Grafiken
- Consensus und Google Scholar
zur Unterstützung der Literaturrecherche
- GitHub Copilot
zur Unterstützung des Schreibprozesses bei der Codegenerierung

Keine KI wurde als eigenständige Quelle verwendet, sondern ausschließlich als unterstützendes Hilfsmittel des Arbeitsprozesses.

Zusammenfassung

Diese Bachelorarbeit beschäftigt sich mit der Entwicklung eines Testframework-agnostischen Verfahrens, welches durch die Entscheidung, welche Testcases für den aktuellen Stand eines Softwareprojektes bereits ausgeführt wurden, und welche nicht, die Laufzeit einer CI-Pipeline bei der Durchführung von Tests reduzieren kann, indem nur Testcases ausgeführt werden, deren Ergebnis noch nicht bekannt ist.

Dazu werden im Rahmen dieser Bachelorarbeit die nötigen Voraussetzungen beleuchtet, um einen möglichst hohen Grad der Framework-Unabhängigkeit zu erreichen. Ebenso wird eine Implementierung des Verfahrens beschrieben, welches das für diesen Zweck erstellte Tool `testAuditor` nutzt. Außerdem werden die Anforderungen an ein Softwareprojekt definiert, die notwendig sind, um das Verfahren in besagtes Projekt integrieren zu können.

Zu Schluss folgt eine Evaluierung, in wie weit die Zielsetzung dieser Bachelorarbeit erfüllt werden konnte.

Begleitet wird diese Arbeit von dem Quellcode von `testAuditor`, einer Demo des Verfahrens in einer exemplarischen Umgebung, einer Benchmark, in welcher die Laufzeiten des Verfahrens mit den Laufzeiten eines regulären Durchlaufs in verschiedenen Testszenarien gegenübergestellt werden, sowie einem Datensatz, in dem die Ausgabeformate einer Auswahl von Testframeworks untersucht werden.

Die entsprechenden Repositorien sind in Kapitel 7 verlinkt.

Inhaltsverzeichnis

Zusammenfassung	iii
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	2
1.3 Begrenzung des Scopes	3
2 Grundlagen und Analyse der Anforderungen	4
2.1 Anforderungen an das Verfahren	4
2.2 Anforderungen an ein Testausgabeformat	5
2.3 Deterministische builds - 'it works on my machine'	6
2.4 Entstehung eines Testausgabeformats	7
3 Vergleich von Testausgabeformaten	11
3.1 Methodik	11
3.1.1 Struktur der Testdateien	12
3.2 Evaluation der häufigsten Formate	14
3.2.1 JUnit XML	14
3.2.2 Test Anything Protocol (TAP)	18
3.2.3 Open Test Reporting (OTR)	20
3.2.4 Evaluation der Anforderungen nach Kapitel 2.2	23
3.2.5 Andere Formate	23
3.3 Fazit zur Evaluation der Testausgabenformate	24
4 Umsetzung	26
4.1 Komponenten des Verfahrens	26
4.1.1 <code>testDiscovery</code>	27
4.1.2 <code>test archive</code>	28
4.1.3 <code>testAuditor</code>	29
4.1.4 <code>testExecution</code>	30
4.1.5 <code>controller</code>	30

4.1.6	user	31
4.2	Anforderungen an ein Testframework	31
4.3	Anforderungen an ein Projekt	31
4.4	Exemplarischer Ablauf	32
5	Evaluation der Bechmark	33
5.1	Aufbau der Benchmark	33
5.1.1	Spezifikationen des ausführenden Systems	34
5.2	Datenerhebung der Benchmark	35
5.3	Ergebnisse der Benchmark	36
5.4	Zusammenfassung der Ergebnisse	38
6	Fazit	40
6.1	Erfüllung der Anforderungen	40
6.2	Möglichkeiten zur Verbesserung	40
	Literaturverzeichnis	41
7	Links	42

1 Einleitung

1.1 Problemstellung

”One cannot “test quality into” a badly constructed software product, but neither can one build quality into a product without test and analysis.”

[1, S. XVII]

Jedes größere Softwareprojekt benötigt eine Vielzahl von Testfällen (fortfolgend als Testcases bezeichnet), um die Qualität der Software zu sichern. Dabei werden die einzelnen Testcases für gewöhnlich in verschiedene Klassen unterteilt (fortfolgend als Testsuites bezeichnet), die verschiedene Abschnitte der Software abdecken. Die meisten gängigen Testframeworks bieten die Möglichkeit, einzelne Testsuites und/oder Testcases gezielt auszuführen, ohne die restlichen Testcases dabei mit auszuführen.

Nehmen wir z.B. einen Entwickler, der bei einem großen Softwareprojekt die Funktionalität einer Datenbank-Schnittstelle anpasst. Testcases, welche die Funktion dieser Schnittstelle abtesten, sind hier sehr relevant - andere Testcases, die sich mit anderen Abschnitten des Projektes befassen, wie etwa UI-Tests oder Tests für Netzwerkverbindungen, sollten bei einem sauber modularisiertem Projekt mit hoher Wahrscheinlichkeit unabhängig von Änderungen der Datenbank-Schnittstelle das selbe Ergebnis wie vor den Änderungen des Entwicklers liefern. Da manche Testcases viel Zeit für ihre Durchführung in Anspruch nehmen, ist es daher sinnvoll, bei der Entwicklung nur einzelne relevante Abschnitte zu testen.

Dennoch muss zur Validierung des Codes regelmäßig jede Testsuite des Softwareprojektes durchlaufen werden, damit durch das partielle Testen des Codes keine Fehler unbemerkt im Projekt verbleiben.

Um dies zu gewährleisten, wird für gewöhnlich auf dem Build-Server nach einem Commit noch einmal die gesamte Testsuite des Projektes durchlaufen, auch wenn manche Testcases möglicherweise bereits ausgeführt wurden, ohne dass sich der Code in der Zwischenzeit geändert hat. Dadurch wird die Laufzeit der

gesamten CI-Pipeline unnötig verlängert.

1.2 Zielsetzung

Ziel dieser Bachelorarbeit ist die Entwicklung eines Lösungsansatzes für die folgende Fragestellung:

Wie kann die Laufzeit von Testphasen in CI-Pipelines unabhängig vom verwendeten Testframework durch die Wiederverwendung bereits vorhandener Testergebnisse reduziert werden, ohne die Aussagekraft der Testergebnisse zu beeinträchtigen?

Zur Beantwortung dieser Frage soll ein Verfahren entwickelt werden, welches anhand der Ergebnisse vergangener Testläufe ermittelt, für welche Testcases noch kein Ergebnis vorliegt, und ausschließlich diese Testcases eine Ausführung veranlasst. Dabei muss zwingend eine komplette Abdeckung aller Testcases erhalten bleiben. Das Ergebnis jedes Testcases muss identisch mit dem Ergebnis bei einem kompletten Neudurchlauf aller Testcases sein, mit Ausnahme von Meta-Daten wie beispielsweise dem Zeitstempel des Testdurchlaufs. Dabei soll die Wahl des Testframeworks möglichst keine Rolle spielen.

Zur praktischen Umsetzung des Verfahrens wird ein Prototyp mit einer beispielhaften Implementation des Verfahrens entwickelt.

Gemessen wird der Erfolg der Entwicklung des Verfahrens anhand einer Evaluation, welche für die genannte Implementation folgende Kennzahlen sammelt:

- Gesamtzahl aller Testcases
- Zahl der vom Verfahren als bereits durchgeführt eingestuften Testcases
- Zahl der vom Verfahren als nicht bereits durchgeführt eingestufte Testcases / durch das Verfahren ausgeführte Testcases
- Laufzeit der Testpipeline unter Verwendung des Verfahrens

- Laufzeit der Testpipeline ohne Verwendung des Verfahrens

Zusätzlich werden die Testergebnisse der Durchläufe mit und ohne Verwendung des Verfahrens miteinander verglichen, um die Gleichwertigkeit der Testergebnisse zu überprüfen.

1.3 Begrenzung des Scopes

Nicht Teil dieser Bachelorarbeit sind die folgenden Punkte:

- Authentizität:

Sämtliche Dateien, welche dem Verfahren von einem Projekt zur Verfügung gestellt werden, werden als legitim betrachtet. Eine Überprüfung der Authentizität oder Autorenschaft einzelner Dateien findet nicht statt.

- Aktualität von Testergebnissen:

Das Verfahren arbeitet mit Testergebnissen, die von einem Akteur innerhalb des Projektes erzeugt werden. Eine Überprüfung, ob die Testergebnisse aus dem aktuellen Stand des Codes entspringen, oder ob der Code oder die Testcases zwischenzeitlich geändert wurden, findet nicht statt.

- Überprüfung der Korrektheit von Daten:

Es wird davon ausgegangen, dass dem Verfahren bereitgestellte Daten und Dateien, wie sie beispielsweise in Kapitel 4.1.1 korrekt und vollständig sind.

2 Grundlagen und Analyse der Anforderungen

In diesem Kapitel werden die Anforderungen an das Verhalten des Verfahrens, die sich aus Kapitel 1.2 ergeben, näher erörtert.

Ebenso werden Grundlagen sowie Anforderungen bezüglich Testausgabeformate vorgestellt.

2.1 Anforderungen an das Verfahren

Folgende Anforderungen werden an einen Durchlauf des Verfahrens gestellt:

- **Die Wahl der verwendeten Testframework(s) ist unerheblich**
Das Verfahren soll framework-agnostisch funktionieren, auch wenn innerhalb des Projektes verschiedene Testframeworks zum Einsatz kommen.
- **Nach Abschluss des Verfahrens wurde jeder Testcase für den aktuellen Stand des Codes mindestens einmal durchgeführt**
Testcases können vor Durchlauf des Verfahrens mehrfach ausgeführt worden sein, allerdings darf während des Verfahrens selbst keinen Testcase ausgeführt werden, der bereits ausgeführt wurde, um die Laufzeit der Testphase so gering wie möglich zu halten. Trotzdem muss die gesamte Testsuite komplett abgedeckt sein, um die Aussagekraft des Testlaufes im Vergleich zu einem komplett neuen Durchlauf nicht zu beeinträchtigen.
- **Die Ausgabe des durch das Verfahren ausgeführten Testlaufs ist auf die selbe Weise verfügbar, wie die anderen Ausgaben**
Die Ausgabe soll in dem selben Format, auf die selbe Art und Weise abgelegt werden, wie auch die anderen Ausgaben, die dem Verfahren zur Verfügung gestellt werden.
- **Die Laufzeit eines Testdurchlaufs unter Verwendung des Ver-**

fahrens ist kleiner oder gleich der Laufzeit eines kompletten Durchlaufs

Das Verfahren soll die Gesamtlaufzeit der Testausführung reduzieren.

2.2 Anforderungen an ein Testausgabeformat

Um ein framework-agnostisches Verfahren zu ermöglichen, muss ein testframework-agnostisches Testausgabeformat gefunden werden, in welchem die Ergebnisse eines Testlaufs in einer standardisierten Form festgehalten werden.

`testAuditor` ist der Kern des Verfahrens, und benötigt zur korrekten Funktion Zugriff auf die Gesamtheit aller Testcases sowie Informationen über den Status von bereits gelaufenen Testcases. Eine detailliertere Beschreibung von `testAuditor` findet sich in Kapitel 4.1.3.

Mit Blick auf diese sowie die unter Kapitel 2.1 beschriebenen Anforderungen an das Verfahren, würde ein ideales Testausgabeformat sämtliche hier gestellten Anforderungen erfüllen:

Ein ideales Ausgabeformat für Tests:

- **ermöglicht die vollständige Ermittlung aller im Projekt vorhandenen Testcases**

Dieser Punkt ist essenziell, zum einen damit `testAuditor` eine Aussage darüber treffen kann, ob eine vollständige Abdeckung aller Testcases erzielt wurde, Zum anderen muss `testAuditor` zum Erzielen einer vollständigen Abdeckung auch auf potentiell jeden vorhandenen Testcase zugreifen können. Dazu muss `testAuditor` selbstverständlich auch jeder Testcase innerhalb des Projektes bekannt sein.

- **identifiziert jeden Testcase eindeutig**

Je nach Testframework und Implementierung ist es gestattet, mehreren Testcases, die in unterschiedlichen Namespaces leben, mit dem selben Namen zu bezeichnen. Aus dem Ausgabeformat muss in jedem Falle eine eindeutige Zuordnung der Testcases und den Ergebnissen möglich sein.

- **sagt für jeden Testcase einzeln aus, ob er ausgeführt wurde oder nicht**

Dieser Punkt setzt die Erfüllung der ersten beiden Punkte voraus. Zusätzlich muss aus dem Eintrag eines Testcases im Ausgabeformat erkennbar sein, ob der Test ausgeführt worden ist.

- **ist ein strukturiertes Format**

Es muss eine erfassbare Struktur für das Testausgabeformats vorhanden sein, damit ein entsprechender Parser für `testAuditor` entwickelt werden kann.

- **ist in vielen Frameworks über viele Programmiersprachen hinweg vertreten**

Es wäre utopisch anzunehmen, dass ein strukturiertes Format existiert, welches von jedem existierendem Testframework gleichermaßen benutzt wird. Dennoch gibt es durchaus gängige Formate, die von unterschiedlichen Testframeworks aus unterschiedlichen Sprachen unterstützt werden, wie folgend in Kapitel 3 untersucht wird.

2.3 Deterministische builds - 'it works on my machine'

Damit in dem hier vorgestellten Verfahren eine qualifizierte Aussage über die Testergebnisse treffen kann, muss gewährleistet sein, dass alle Testcases unabhängig vom ausführenden System immer das selbe Testergebnis liefern. Beispielsweise muss ein Testcase, der auf der Machine eines Entwicklers ein bestimmtes Ergebnis liefert, muss auch zwingend auf einem Build-Server das selbe Ergebnis liefern. Andernfalls könnte eine Diskrepanz zwischen den Testergebnissen zu einer verfälschten Aussage des Verfahrens über den aktuellen Stand der Tests führen.

Im Rahmen dieser Bachelorarbeit wird hierfür der Linux-package manager nix

benutzt. Nix bietet die Funktionalität, die Entwicklungsumgebung deklarativ zu beschreiben, indem benötigte Abhängigkeiten in einer sogenannten flake festgelegt werden. Eine dazugehörige Datei flake.lock enthält dann detaillierte Informationen über den build. Dadurch können auf verschiedenen Systemen identischen Entwicklungsumgebungen aufgebaut werden.

Es gilt zu beachten, dass durch diesen Aufbau nur explizit deklarierte interne Abhängigkeiten deterministisch erfasst werden können. Externe Faktoren wie etwa Netzwerkschnittstellen, zufallbasierte Ereignisse oder Ähnliches können auf diese Weise nicht reproduzierbar festgehalten werden.

Ein vollständige Garantie des Determinismus zwischen Systemen ist nur schwer zu erreichen, jedoch bietet Nix hier eine ausreichende Grundlage.

2.4 Entstehung eines Testausgabeformats

Um zu verdeutlichen, wie die finale Form Testausgabeformates entsteht, wird in diesem Kapitel auf die näher auf die Faktoren eingegangen, welche die genaue Struktur und das finale Aussehen des Testausgabeformates beeinflussen. Die Form eines Testausgabeformates hängt im Wesentlichen von den folgenden Faktoren ab, deren Aufteilung der in [2] vorgestellten Struktur eines Testframeworks folgt:

- **Struktur der zugrunde liegenden Testdateien:**

Die (Nicht-)Verwendung von Klassen, Testsuites oder anderen vom Testframework oder der Programmiersprache bereitgestellten Optionen zur Organisation der Testcases, sowie die Dateistruktur der Testdateien können einen Einfluss auf das Endresultat haben

- **verwendetes Testframework**

- **Test Runner:**

Auch als *Scheduler*, *Harness* oder *Orchestrator* bezeichnet, beschreibt der Begriff "Test Runner" hier die Komponente eines Testframeworks,

welche die Testcases ausführt, und die Ergebnisse an den Test Reporter weiterreicht. Der Test Runner beeinflusst beispielsweise die Darstellung von Ereignissen, die während der Ausführung der Testsuite auftreten und in die Ausgabe übernommen werden.

- **Test Reporter:**

Auch als *test generator* bezeichnet, beschreibt "Test Reporter" hier jene Komponente eines Testframeworks, welche aus dem vom Test Runner bestimmten Ergebnis der Testsuite die menschen- bzw. maschinenlesbare Ausgabe generiert.

- **verwendetes Testausgabeformat:**

Damit ist sowohl das Dateiformat, als auch die Wahl eines strukturierten Ausgabeformats gemeint. Näheres dazu in Kapitel 3.

In den meisten Testframeworks übernimmt das Testframework selbst die Aufgaben des Test Runners und des Test Reporters, in manchen Fällen werden diese Komponenten aber auch durch andere Prozessen bereitgestellt. Dies ist zum Beispiel in der Programmiersprache Java der Fall: Hier stellen die beiden build-tools maven und gradle jeweils ihre eigenen Test Runner bzw. Test Reporter zur Verfügung, was unterschiedliche Kombinationen aus Testframework, Test Runner und Test Reporter erlaubt. Dass dies auch zu Unterschieden in der Testausgabe führen kann, ist in Codeblock 2.1 dargestellt.

JUnit5, Jupiter:

```
<?xml version="1.0" encoding="UTF-8"?>
<testsuite name="JUnit Jupiter" tests="5" skipped="1" failures="1"
  errors="0" time="0.036" hostname="DESKTOP-I9APAL0"
  timestamp="2026-07-04T00:45:01">
<properties>
<property name="file.encoding" value="UTF-8"/>
<property name="file.separator" value="/" />
...
<testcase name="test_neg()" classname="BasicTest" time="0.006">
```

JUnit5, Jupiter mit maven surefire:

```
<?xml version="1.0" encoding="UTF-8"?>
<testsuite xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation=
    "https://maven.apache.org/surefire/maven-surefire-plugin/xsd/surefire-test-report.xsd"
  version="3.0.2" name="BasicTest" time="0.013" tests="4"
  errors="0" skipped="1" failures="1">
  <properties>
    <property name="java.specification.version" value="21"/>
    <property name="sun.jnu.encoding" value="UTF-8"/>
    ...
  <testcase name="test_neg" classname="BasicTest" time="0.0"/>
  ...
```

JUnit5, Jupiter mit gradle test task:

```
<?xml version="1.0" encoding="UTF-8"?>
<testsuite name="BasicTest" tests="4" skipped="1" failures="1"
  errors="0" timestamp="2026-07-03T22:47:00.701Z"
  hostname="DESKTOP-I9APAL0" time="0.009">
  <properties/>
  <testcase name="test_neg()" classname="BasicTest" time="0.001"/>
  ...
```

Codeblock 2.1: direkter Vergleich von JUnit XML, erzeugt durch JUnit5 mit Jupiter in Java. Jeder Ausgabe liegen die selben beiden Testdateien zu Grunde.
... bezeichnet gekürzte Stellen.

Obwohl für jede Ausgabe JUnit5 mit Jupiter als Testframework benutzt wird, jede Ausgabe JUnit XML als Format aufweist, und jede Ausgabe auf den selben beiden Testdateien basiert, gibt es durch die unterschiedlichen Test Runner und Test Reporter deutliche Unterschiede in der Testausgabe.

So ignoriert gradle beispielsweise in diesem exemplarischen Fall das tag `<properties>`, während maven und Jupiter (ohne build-tool) eine exessive Liste von `<property>` in der Testausgabe mitgeben.

Auch die Benennung der Testcases unterscheidet sich:

Jupiter und gradle behalten den Namen `test_neg()` bei, so wie er auch in der Testdatei verwendet wird, während maven den Namen zu `test_neg` kürzt.

Ähnliches bei der Benennung der Testsuite: Da gradle und maven für jede Datei eine eigene Ausgabedatei erzeugen, benutzen die beiden build-tool den Namen der jeweiligen Testsuite, hier "BasicTest", als Wert des Attributes "name" im tag `<testsuite>`. Jupiter gibt dagegen nur eine Ausgabedatei aus, die das Ergebnis aller Testdateien enthält, und benutzt in dieser den Standardwert "Junit Jupiter" als Namen für `<testsuite>`.

Es ist also ersichtlich, dass

3 Vergleich von Testausgabeformaten

Um das Tool `testAuditor` zu entwickeln, welches framework-agnostisch eingesetzt werden kann, muss `testAuditor` die Ausgabe eines Testlaufes unabhängig vom verwendeten Framework parsen können. Die Anforderungen an ein Testausgabeformat wurden bereits in Kapitel 2.2 untersucht.

Standartmäßig geben die meisten Testframeworks das Ergebnis eines Testlaufs in einem unstrukturiertem Format über die Konsole aus. Allerdings bieten fast alle etablierten Testframeworks auch eine Auswahl an Dateiformaten an, über welche die Testausgabe ebenfalls präsentiert werden kann.

Für diese Dateiformate wurden über die Jahre einige framework-übergreifende Ausgabeformate entwickelt, die sich in unterschiedlicher Ausprägung durchsetzen konnten.

Es gilt nun, ein geeignetes Testausgabeformat zu finden, dass von `testAuditor` als Quelle für Testergebnisse genutzt werden kann.

3.1 Methodik

Aufgrund der Vielzahl von verschiedenen Testframeworks und der Vielfältigkeit ihrer Ausgabeformate können im Rahmen dieser Bachelorarbeit nur die verbreitesten Formate untersucht werden.

Zur Untersuchung der praktischen Umsetzung einzelner Formate und zur exemplarischer Erfassung ihrer Verbreitung wird ein limitierter Datensatz betrachtet. Dabei gilt zu beachten, dass bei diesem Datensatz nur eine kleine Menge ausgewählter Programmiersprachen beinhaltet. Außerdem beschränken sich die untersuchten Testframeworks auf die am weitesten verbreiteten in ihrer jeweiligen Sprache, um die Etablierung von Testausgabeformaten möglichst praxisnahe darstellen zu können. Außerdem bieten länger etablierte Testframeworks erfahrungsgemäß eine breitere Auswahl von Testausgabeformaten, als weniger verbreitete oder neue Testframeworks.

Bei der Untersuchung von Plugins für zusätzliche Testausgabeformate wurden ausschließlich Plugins berücksichtigt, welche über den hier benutzten package

manager nix bereitgestellt werden. Würden externe Plugins, welche zumeist aus unabhängig gepflegten Repositorien stammen, ebenfalls betrachtet werden, würde die Zahl der unterstützten Testausgabeformate bei den meisten untersuchten Testframeworks deutlich steigen. Da keine Aussage über die Vertrauenswürdigkeit und Qualität der externen Plugins getroffen werden kann, beschränkt sich diese Untersuchung auf von nix bereitgestellte Plugins.

3.1.1 Struktur der Testdateien

Die Struktur der Testdateien wird für jede untersuchte Programmiersprache soweit möglich identisch gehalten.

Es existieren zwei Testdateien. Sofern das Testframework die Möglichkeit bereitstellt, besitzt jede der Dateien ihren eigenen Namespace. Die Bezeichnung der Namespaces kann variieren, da manche Testframeworks eine bestimmte Namenskonvention voraussetzen.

In der ersten Datei liegen 4 Testcases im Namespace `BasicTest`:

- `test_neg()`: Ein einfacher Testcase, der eine korrekte Assertion beinhaltet und nicht fehlschlägt.
- `test_pos()`: Ein einfacher Testcase, der eine falsche Assertion beinhaltet und fehlschlägt.
- `test_skip()`: Ein Testcase, der übersprungen wird. Manche Testframeworks unterscheiden zwischen übersprungenen, und ignorierten Testcases: Übersprungene Testcases werden nicht ausgeführt, aber als übersprungen in der Testausgabe markiert. Ignorierte Testcases werden weder ausgeführt, noch in die Testausgabe übernommen. Dies macht ignorierte Testcases für die Evaluation der Testausgabeformate uninteressant.
- `test_same_name()`: Ein einfacher Testcase, der eine korrekte Assertion beinhaltet und nicht fehlschlägt. In der zweiten Datei existiert ein Testcase mit dem selben Namen.

In der zweiten Datei liegt ein Testcase im Namespace `BasicTest2`:

- `test_same_name()`: Ein einfacher Testcase, der eine korrekte Assertion beinhaltet und nicht fehlschlägt. In der ersten Datei existiert ein Testcase mit dem selben Namen.

`test_same_name` untersucht, ob es allein aus der Testausgabe heraus möglich ist, einen Testcase eindeutig zu indentifizieren, auch wenn in unterschiedlichen Namespaces verschiedene Testcases den selben Namen aufweisen.

Tabelle 1 zeigt die Ergebnisse des Datensatzes zur Untersuchung der Verbreitung von Testausgabeformaten.

Framework	Sprache	JUnit XML	XML (anderes)	OTR	TAP	JSON	HTML
catch2	C++	✓	✓	✗	✓	✗	✗
doctest	C++	✓	✓	✗	✗	✗	✗
gtest	C++	✓	✗	✗	✗	✓	✗
hspec	Haskell	~	✗	✗	✗	✗	✗
tasty	Haskell	~	✗	✗	~	✗	✗
junit5/jupiter	Java	✓	✗	✓	✗	✗	✗
junit5/jupiter (maven surefire)	Java	✓	✗	~	✗	✗	~
junit5/jupiter (gradle test task)	Java	✓	✗	~	✗	✗	✓
behat	PHP	✓	✗	✗	✗	✓	✗
codeception	PHP	✓	✗	✗	✗	✗	✓
phpunit	PHP	✓	✗	✓	✗	✗	✓
pytest	Python	✓	✗	✗	~	~	~
unittest	Python	✓	✗	✗	✗	✗	✗
Von 13 Gesamt (Support nativ oder per Plugin)		13	2	4	3	3	5

✓ Nativ unterstützt
~ Über Plugins unterstützt
✗ Nicht offiziell unterstützt

Tabelle 1: Übersicht der Verbreitung bestimmter Formate. Sofern nicht anders angegeben, werden die vom jeweiligen Testframework bereitgestellten Test Runner / Test Reporter benutzt.

3.2 Evaluation der häufigsten Formate

In diesem Kapitel werden die häufigsten frameworkübergreifend verwendeten Testausgabeformate kurz vorgestellt und nach den in Kapitel 2.2 festgesetzten Kriterien auf ihre Eignung zur Benutzung als Input für `testAuditor` evaluiert.

3.2.1 JUnit XML

Übersicht

JUnit XML ist ein XML-basiertes Format, welches ursprünglich für das Java-Testframework JUnit entwickelt wurde. Für JUnit XML existiert kein einheitlicher Standard, d.h. es existiert auch kein beschreibendes .xsd Schema, wie sonst für XML-Formate üblich. Trotzdem wurde JUnit XML von vielen Testframeworks in gleicher oder ähnlicher Form adaptiert, wie es ursprünglich für JUnit entworfen wurde. Außerdem existieren viele weitere Schnittstellen zwischen JUnit XML und anderen für das Software-Testing relevante Programme, beispielsweise viele CI/CD-Systeme. Auf diese Weise hat sich JUnit XML zum häufigsten benutzte Testausgabeformat für Testframeworks entwickelt.

Struktur

Wie bereits erwähnt gibt es bei JUnit XML keine standardisiertes Schema. Allerdings hat sich eine Grundstruktur durchgesetzt, die von nahezu allen Testframeworks, welche JUnit XML als Testausgabeformat anbieten, verwendet wird. Ein typisches Beispiel für diese Grundstruktur ist Codeblock 3.1:

```
<?xml version="1.0" encoding="UTF-8"?>
<testsuites name="default">
  <testsuite name="Basic tests" file="features/basic.feature"
    tests="4" skipped="0" failures="2" errors="0" time="0.006">
    <testcase name="test_neg" classname="Basic tests"
      status="passed" time="0.004" file="features/basic.feature"
      line="4"></testcase>
    ...
  </testsuite>
</testsuites>
```

Codeblock 3.1: JUnit XML, erzeugt durch das PHP-Testframework behat.
... bezeichnet gekürzte Stellen.

Das root-element `<testsuites>` enthält eine Menge von Elementen mit der Bezeichnung `<testsuite>`, welche jeweils eine Menge von Elementen mit der Bezeichnung `<testcase>` enthalten. Jedes `<testcase>` ist korrespondierend zu einem Testcase aus der ursprünglichen Testsuite des zu testenden Programms und enthält für gewöhnlich Informationen über das Testergebnis, den Ablauf, und den ursprünglichen Testcase. Hier ist allerdings wieder wichtig zu erwähnen, dass der Inhalt dieses Elements ebensowenig standartisiert ist, wie der Inhalt der übrigen Elemente. Deutlich wird dieser Sachverhalt in Codeblock 3.2. Unterschiede in der grundlegenden Struktur zeigen sich z.B. in Codeblock 3.3. Hier wird das Element `<testsuite>` verschachtelt verwendet, statt wie sonst üblich nur einmal.

```
junit5 (maven), Java:
<testcase name="test_neg" classname="BasicTest" time="0.012"/>

phpunit, PHP:
<testcase name="test_neg" file="/home/.../test.php" line="7"
    class="BasicTest" classname="BasicTest" assertions="1"
    time="0.000645"/>

gtest, C++:
<testcase name="test_neg" file="./test/gtest/test.cpp" line="3"
    status="run" result="completed" time="0."
    timestamp="2026-06-29T22:44:17.561" classname="BasicTests" />
```

Codeblock 3.2: direkter Vergleich verschiedener Implementierungen des `<testcase>`-tags in ausgewählten JUnit-XML Ausgaben. Verglichen wird jeweils die Ausgabe des fehlerlos ausgeführten Testcases `test_neg`.
... bezeichnet gekürzte Stellen.

```
<?xml version='1.0'?>
<testsuites errors="0" failures="1" tests="5" time="0.001">
    <testsuite name="AllTests" tests="5">
```

```

<testsuite name="Test1" tests="4">
    <testcase name="test_neg" time="0.000"
        classname="AllTests.Test1" />
    ...

```

Codeblock 3.3: JUnit XML, erzeugt durch das Haskell-Testframework tasty mit dem ant-Plugin.
... bezeichnet gekürzte Stellen.

Aus Codeblock 3.2 ist ersichtlich, dass sich die benutzten Attribute von `<testcase>` sowie deren Inhalt stark unterscheiden können. Jedoch werden zwei bestimmte Attribute von den meisten Testframeworks gesetzt, und auch in gleicher Weise verwendet:

- `name=''` enthält den Namen des Testcase
- `classname=''` enthält die Bezeichnung des Namespaces bzw. der Klasse oder Gruppe des Testcase

Dieser Trend aus Codeblock 3.2 setzt sich auch in den übrigen im Datensatz untersuchten JUnit XML Ausgaben fort. Es gibt zwar einige wenige Ausnahmen wie exemplarisch in Codeblock 3.4 dargestellt, diese sind jedoch eher selten. Eine ausführliche Übersicht, wie häufig die eindeutige Zuordnung zwischen Testergebnis und Testcase durch die Attribute `name` und `classname` möglich ist, ist in Tabelle 2 zu finden.

```

<testcase name="test_neg" class="BasicTest"
    file="/home/.../Test.php" time="0.000252" assertions="1"/>

```

Codeblock 3.4: JUnit XML für den Testcase `test_neg`, erzeugt durch das PHP-Testframework codeception. Codeception benutzt statt dem häufig verwendeten Attribut `"classname"` das Attribut `"class"`.
... bezeichnet gekürzte Stellen.

Framework	Sprache	Zuordnung über name+classname	Zuordnung über alternativen Attribute	Namen der alternativen Attribute
catch2	C++	✓	~	~
doctest	C++	✓	~	~
gtest	C++	✓	~	~
hspec	Haskell	✓	~	~
tasty	Haskell	✓	~	~
junit5	Java	✓	~	~
junit5	Java (maven)	✓	~	~
junit5	Java (gradle)	✓	~	~
behat	PHP	✓	~	~
codeception	PHP	✗	✓	name+class
phpunit	PHP	✓	~	~
pytest	Python	✓	~	~
unittest	Python	✓	~	~

- ✓ Eindeutige Zuordnung möglich
- ~ nicht notwendig
- ✗ Eindeutige Zuordnung nicht möglich

Tabelle 2: Übersicht zur eindeutigen Zuordnung des Testergebnis zum ursprünglichen Testcase über Attribute in JUnit XML

Verbreitung

Da JUnit XML ursprünglich spezifisch für das Testframework Junit entwickelt wurde, folgt die grundlegende Struktur den Gegebenheiten und Anforderungen dieses Frameworks. Zwar folgen viele weitere Testframeworks auch in anderen Sprachen einer ähnlichen Struktur oder können die JUnit XML Struktur trotz einiger Unterschiede benutzen, in einzelnen Fällen kommt es jedoch zu Schwierigkeiten und ungewöhnlichen Anpassungen der XML-Struktur.

Abgesehen davon ist JUnit XML das einzige Testausgabeformat, dass von jedem im Datensatz untersuchten Testframework ausgegeben werden kann. Zumeist sogar nativ, vereinzelt muss die Unterstützung erst über offiziell unterstützte Plugins hinzugefügt werden.

Fazit zu JUnit XML

Aufgrund der weiten Verbreitung und der trotz der fehlenden Standardisierung ähnlichen Umsetzung in vielen Testframeworks, ist JUnit XML ein geeigneter In-

put für `testAuditor`, da es in der Praxis trotz einiger potentieller Fehlerquellen mit einer Vielzahl von Testframeworks kompatibel ist.

3.2.2 Test Anything Protocol (TAP)

Übersicht

Das Test Anything Protocol wurde ursprünglich für die Programmiersprache Perl entworfen, wurde über die Jahre hinweg jedoch auch für Testframeworks in anderen Sprachen weiterentwickelt und integriert. Anders als bei JUnit XML wird für TAP ein standardisiertes Schema bereitgestellt und gepflegt, siehe [3]. Anders als die meisten anderen hier untersuchten Testausgabeformate ist TAP keine spezialisierte Struktur eines bereits bestehenden Dateiformates, sondern ein eigenständiges Dateiformat.

Struktur

Zu Beginn einer Datei wird die Gesamtzahl aller Testcases der Datei angegeben. Danach folgt eine Auflistung der Testergebnisse mit optionalen Zusatzinformationen, zumeist der Name des Testcase. Übersprungene Tests können über zusätzliche Dekoratoren idikatiert werden, die in TAP als "Derectives" bezeichnet werden.

Da die meisten Zusatzinformationen optional sind, zeigt Codeblock 4.1 bereits eine vollständig gültige, minimale TAP-Datei:

```
1..1
ok
```

Codeblock 3.5: Minimales TAP-Beispiel

Solch eine Darstellung ist natürlich ungeeignet, da so keine Zuordnung zwischen Testergebnis und Testcase möglich ist. In der Praxis kann dieser Zusammenhang allerdings durch die optionalen Zusatzinformationen gesesetzt werden, wie beispielhaft mit pytest in Codeblock 3.6 demonstriert:

```
# TAP results for test/pytest/test.py
ok 1 test/pytest/test.py::test_neg
not ok 2 test/pytest/test.py::test_pos
...
ok 3 test/pytest/test.py::test_skip # SKIP ...
ok 4 test/pytest/test.py::test_same_name
1..4
```

Codeblock 3.6: TAP, erzeugt durch pytest.
... bezeichnet gekürzte Stellen.

Eignung

Eine eindeutige Zuordnung zwischen Testergebnis und Testcase ist nicht in allen untersuchten Testframeworks, die TAP als Ausgabeformat bieten, möglich. Nehmen wir catch2: Zwei Testscases dürfen in diesem Framework den selben Namen aufweisen, solange sie mit unterschiedlichen Labels getaggt sind. Diese Zusatzinformationen sind allerdings nicht zwingend im TAP-Format erforderlich, und werden vom catch2-Reporter schlicht nicht übernommen. Dies kann eine Ausgabe wie in Codeblock 3.7 zur Folge haben, bei der zwei Testergebnisse mit dem selben Namen bezeichnet werden:

```
...
# test_same_name
not ok 3 - false
# test_same_name
ok 4 - true
...
```

Codeblock 3.7: TAP, erzeugt durch catch2.
... bezeichnet gekürzte Stellen.

Dies macht TAP in diesem Falle als Ausgabeformat ungeeignet. Andere Testframeworks wie beispielsweise pytest oder tasty sorgen allerdings für eine eindeutige Zuordnung in ihren jeweiligen Ausgaben.

Fazit zu TAP

TAP ist aufgrund seiner sehr geringen Minimalstruktur und vor allem wegen seiner nur mäßigen Verbreitung eher nicht als Input für `testAuditor` geeignet.

3.2.3 Open Test Reporting (OTR)

Übersicht

Open Test Reporting (OTR) wird von dem selben Entwicklerteam, welches auch hinter dem Framework JUnit und damit auch hinter JUnit XML steht, als Nachfolger von eben diesem entwickelt. Dabei haben sie laut eigenen Angaben das Ziel, mit OTR die historisch gewachsenen Probleme von JUnit XML (siehe Kapitel 3.2.1) zu umgehen. OTR soll nach Vorstellungen der Entwickler auf lange Sicht JUnit XMLs Platz als häufigstes Ausgabeformat für Testläufe einnehmen. [4]

Anders als die anderen vorstellten Formate gibt es bei OTR zwei mögliche Dateiformate: Es gibt Implementierungen für XML und HTML. Beide Implementierungen sind im Gegensatz zu JUnit XML standardisiert und folgen einem definiertem Schema. Beim XML-Format wird zwischen zwei Strukturen unterschieden: Hierarchisch und eventbasiert. Beide Formate nutzen das selbe Set von Kernelementen, unterscheiden sich aber in ihrer Struktur. Für beide Strukturen gibt es Programmiersprachen-spezifische Erweiterungen.

Das HTML-Format wird ausgehend von einer vorher erzeugten OTR-XML Datei erzeugt, und ist ebenso erweiterbar.

Struktur

Für diese Untersuchung wird nur die eventbasierte XML-Struktur betrachtet, da diese Struktur das von den im Datensatz untersuchten Testframeworks benutzte Format ist.

Im event-basiertem Format werden die Ergebnisse nicht wie in JUnit XML oder TAP durch ein einzelnes Element mit optionalen Attributen repräsentiert,

sondern über verschiedene Elemente, welche jeweils das Ergebnis eines Event der Testausführung enthalten. Demonstriert ist diese Struktur in Codeblock 3.8:

```
...
<e:started id="3" parentId="2" name="test_neg"
    time="2026-06-25T14:11:44.985961Z">
...
<e:finished id="3" time="2026-06-25T14:11:44.986266Z">
...
```

Codeblock 3.8: OTR von `test_neg`, erzeugt durch `phpunit`.
... bezeichnet gekürzte Stellen.

Hierbei enthält das Element `<e:started>` Informationen über den Testcase aus der ursprünglichen Testdatei, während `<e:finished>` Informationen über das Ergebnis des Testlaufs enthält.

Wie auch in JUnit XML und TAP sind die meisten Informationen und Attribute optional, mit einem entscheidenden Unterschied: Aus dem Schema von OTR lässt sich entnehmen, dass die Elemente `<e:started>` und `<e:finished>` folgende Attribute aufweisen müssen: `id`, `time` und `name`. Während `id` intern in der Ausgabe benötigt wird, um die verschiedenen Events einander zuzuordnen, und `time` lediglich den Zeitpunkt des Beginns des Events festhält, ist für die Eignung von OTR als Input für `testAuditor` vor allem `name` interessant: Dieses Attribut enthält den Namen des Testcase. Jedoch genügt wie bereits bei TAP in Codeblock 3.7 gezeigt das Vorhandensein des bloßen Namens nicht zur eindeutigen Identifizierung eines Testcase. Die dazugehörige Suite wird in OTR innerhalb des Elements `<metadata>` angegeben. Dieses Element ist jedoch optional. Somit ist die eindeutige Zuordnung von Testcases zu ihrem Ergebnis zwar teilweise in der Struktur festgehalten, aber noch nicht ausreichend garantiert.

Eignung

Da OTR zum Zeitpunkt der Niederschrift dieser Bachelorarbeit noch sehr jung ist und aktiv entwickelt wird, bleibt abzuwarten, ob die Entwickler ihr Ziel eines standardisierten, Programmiersprachen-agnostischen Testausgabeformates erreichen können.

Außerdem ist die Verbreitung von OTR noch sehr limitiert: Von den in Tabelle 1 betrachteten Testframeworks sind lediglich phpunit und junit5 (welches, wie eingangs erwähnt, vom selben Entwicklerteam wie OTR betreut wird) mit jupiter als runner in der Lage, OTR nativ zu erzeugen. Außerdem kann junit5 mit den runnern von maven/gradle zumindest über Plugins eine OTR-Ausgabe erzeugen.

Fazit zu OTR

Auch wenn OTR die am besten geeignetste Struktur der bisher untersuchten Testausgabeformate bietet, ist die Verbreitung von OTR zum aktuellen Zeitpunkt noch zu gering, um die framework-agnostik des Verfahrens gewährleisten zu können, sollte es als Input für `testAuditor` genutzt werden.

3.2.4 Evaluation der Anforderungen nach Kapitel 2.2

Anforderung	JUnit XML	TAP	OTR
ermöglicht Ermittlung aller vorhandenen Testcases	✗	✗	✗
eindeutige Identifizierung von Testcases	~	~	~
sagt für jeden Testcase einzeln aus, ob er ausgeführt wurde oder nicht	~	~	~
strukturiertes Format	~	✓	✓
weite Verbreitung	✓	~	~

✓	Erfüllt
~	In Teilen erfüllt
✗	Nicht erfüllt

Tabelle 3: Evaluation der Testausgabeformate nach den in Kapitel 2.2 definierten Anforderungen

3.2.5 Andere Formate

Neben den bisher untersuchten ganz oder teilweise strukturierten Formaten bieten viele Testframeworks auch unstrukturierte Ausgaben in unterschiedlichen Dateiformaten an. Konkret wurden bei der Untersuchung des Datensatzes Testausgaben in den folgenden Dateiformaten festgestellt:

- XML (ohne JUnit XML oder OTR)
- HTML (ohne OTR)
- JSON

Das Problem bei diesen Testausgaben ist, dass ihre Struktur voll und ganz von dem Testframework abhängig ist, von dem sie erzeugt werden.

JSON, von pytest in python erzeugt:

```
{"created": 1782247688.1675658,
```

```

"duration": 0.024770736694335938,
"exitcode": 1,
"root": "/home/.../python/src",
"environment": {},
"summary": {
    "passed": 3,
    "failed": 1,
    "skipped": 1,
    "total": 5,
    "collected": 5
},
"collectors": [
...
JSON, von behat in php erzeugt:
{"tests": 5,
"skipped": 0,
"failed": 2,
"pending": 0,
"undefined": 0,
"time": 0.011,
"suites": [
...

```

Codeblock 3.9: Vergleich von der JSON-Ausgabe von pytest und behat.
... bezeichnet gekürzte Stellen.

Für diese von Grund auf verschiedenen Ausgaben kann offensichtlich kein einheitlicher Parser geschrieben werden.

Somit können für die Wahl des Inputs für `testAuditor` nur die vorangegangenen untersuchten Testausgabeformate in Betracht gezogen werden.

3.3 Fazit zur Evaluation der Testausgabenformate

<https://xkcd.com/927/>

TODO: Überarbeiten.

Anhand der Evaluation der unterschiedlichen Testausgabeformate in Kapitel 3 wird deutlich, dass es keinen eindeutigen Kandidaten für die Auswahl des Testausgabeformats, welches alle in Kapitel 2.2 gelisteten Anforderungen erfüllt. Ein gemeinsames Problem: zeigen keine nicht-ausgeführten tests an

4 Umsetzung

4.1 Komponenten des Verfahrens

Das vorgestellte Verfahren besteht aus den folgenden Komponenten:

- **testDiscovery**: liefert die Gesamtheit aller Testcases
- **test archive**: Enthält die Ausgabe aller Testdurchläufe, die für den aktuellen Stand beachtet werden sollen
- **testAuditor**: Evaluiert, welche Testcases noch ausgeführt werden müssen
- **testExecution**: Führt eine gegebene Menge von Testcases aus
- **controller**: Steuert die Interaktion mit dem user, sowie die Kommunikation von Komponenten untereinander
- **user**: Interagiert nach eigenem Ermessen mit dem manager

Eine Übersicht zur Interaktion der einzelnen Komponenten liefert Abbildung 1:

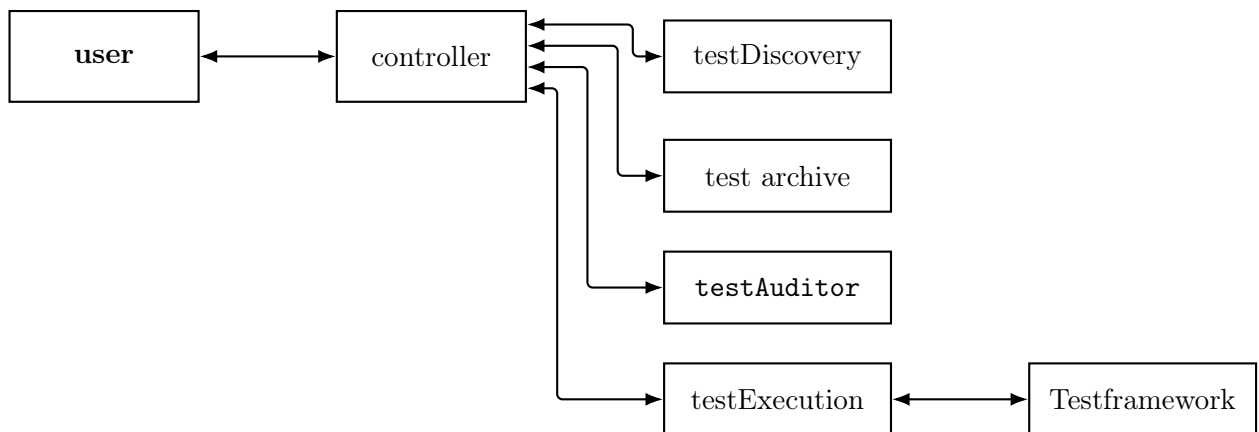


Abbildung 1: Schematische Darstellung der Interaktion von user, controller, testAuditor, Hilfskomponenten und dem Testframework.

Im Folgenden werden die einzelnen Komponenten des Verfahrens, so wie ihr

Zusammenspiel untereinander im Detail erläutert.

4.1.1 testDiscovery

Durch die Wahl für JUnit XML als Ausgabeformat, das vom Verfahren verstanden werden soll, müssen die in Kapitel ?? aufgelisteten Anforderungen, die von JUnit XML nicht erfüllt werden, anderweitig umgesetzt werden, damit alle in Kapitel 2.1 gesetzten Ziele erreicht werden können.

Die größte Herausforderung ist, dass aus einer JUnit-XML Ausgabe nur jene Testcases ermittelt werden können, welche während des Testlaufes ausgeführt wurden. `testAuditor` benötigt jedoch zwingend Informationen über die Gesamtheit aller Testcases im Projekt.

Diesem Zweck dient die `testDiscovery`:

Bei dieser Komponente ist die Implementierung als shellscript am geeignetsten, da sie bei einem Aufruf die Gesamtheit aller Testcases des Projektes in einem definierten Format zurückgibt. Manche Testframeworks bieten Zugriff auf ihre interne Discovery für Testcases, was in diesem Fällen die Implementation recht einfach gestaltet.

Format der Ausgabe der testDiscovery

`testAuditor` erwartet die Ausgabe der `testDiscovery` in diesem Format:

```
<?xml version="1.0" encoding="utf-8"?>
<testsuite>
  <testcase classname="" name="" qualifiedName=""/>
  <testcase classname="" name="" qualifiedName=""/>
</testsuite>
```

Codeblock 4.1: Format der `testDiscovery`

Der Inhalt der beiden tags "classname" und "name" für einen Testcase muss dabei zwingend identisch mit dem Inhalt der beiden tags sein, wie er in der JUnit-XML Ausgabe des Testcase steht, da in einem späterem Schritt über

die Kombination dieser beiden tags die Testcases eindeutig im JUnit XML identifiziert werden.

`qualifiedName` enthält eine Zeichenfolge, über die das Testframework einen Testcase identifizieren, und ausführen kann. Je nach Testframework werden dazu der Name des Testcase, Dateipfade, Namen von klassen oder Testsuites und/oder weitere Informationen benötigt. `qualifiedName` dient in dem Schritt der `testExecution` (siehe Kapitel 4.1.4) bei der Ausführung einzelner Testcases.

Die `testDiscovery` ist hochgradig vom verwendeten Testframework abhängig, da die spezielle Implementierung der Testausführung des jeweiligen Testframeworks beachtet werden muss. Daher muss diese Komponente für jedes Softwareprojekt eigens angepasst werden.

4.1.2 test archive

Das `test archive`, kurz `archive`, beschreibt die Komponente des Verfahrens, welche die in JUnit XML festgehaltenen Testausgaben von vorherigen Testläufen beinhaltet, die `testAuditor` als Input zur Verfügung stehen.

Ein Projekt muss so konfiguriert werden, dass sichergestellt ist, dass keine Dateien aus dem `archive`, die sich auf einen alten Stand des Projekts beziehen, vom Verfahren genutzt werden. Hierfür bietet sich die Implementierung des `archive` über git notes an, wie im folgenden Abschnitt untersucht wird.

git notes

Eine wenig bekannte und selten genutzte Funktionalität von Git ist, dass an einen Commit sogenannte git notes angehängt werden können. Der Inhalt dieser notes wird separat vom eigentlichen Inhalt des Commits verwaltet, der Hash eines Commits wird nicht durch Veränderungen der notes beeinflusst. Dadurch können einem Commit zusätzliche Informationen hinzugefügt werden, ohne die Historie des Repositories zu verändern.

Dadurch bieten sich git notes in zweierlei Hinsicht für die Rolle des `test archive` an:

Zum einen wird das eigentliche Projekt nicht durch zusätzliche Dateien vom Verfahren verändert. Zum anderen gehört jede note zu genau einem Commit, wodurch vermieden werden kann, dass das Verfahren Dateien aus dem `archive` für einen vorherigen Commit erhält, da die notes für jeden Commit neu erstellt werden müssen.

4.1.3 testAuditor

Die Funktionalität von `testAuditor` als Kern des Verfahrens lässt sich am Besten mit einer einfachen Mengengleichung beschreiben:

Sei T_{ges} die Menge aller Testcases eines Projektes, T_{run} die Menge aller Testcases, für die ein Ergebnis vorliegt. `testAuditor` berechnet $T_{ges} - T_{run} = T_{todo}$, wobei T_{todo} die Menge aller Testcases ist, für die noch kein Ergebnis vorliegt. Da nach Kapitel 1.3 von der Korrektheit und Vollständigkeit von T_{run} ausgegangen werden darf, ist trivial, dass $T_{run} \subset T_{ges}$, sowie $T_{todo} \subset T_{ges}$.

T_{ges} wird von `testDiscovery` bereitgestellt, während das `test archive` T_{run} enthält. Beide Datenmengen werden `testAuditor` vom `controller` über ein in Kapitel 4.1.3 definiertes Format übermittelt.

Zu guter Letzt gibt `testAuditor` eine Menge von `qualifiedNames` an den `controller` zurück. Diese Menge beschreibt T_{todo} .

Format zur Kommunikation mit testAuditor

-> TODO: Format beschreiben

Parser für JUnit XML

Das Parsen des vorangehend beschriebenen Kommunikationsformat, sowie des in Kapitel 4.1.1 beschriebenen Format der `testDiscovery` ist trivial.

Interessanter ist die Umsetzung eines Parsers für JUnit XML für die Daten aus dem `test archive`. Wie in Kapitel 3.2.1 behandelt, besitzt JUnit XML kein standardisiertes Schema. Demensprechend muss ein Parser für solche Dateien auf eine Art und Weise konfiguriert werden, die ohne eine festes erwartbare Struktur auskommt. Dafür wird bei der Implementation des Parsers ein

streaming-basierter Ansatz verwendet:

Die XML-Datei wird sequenziell Element für Element eingelesen. Sobald ein Element mit dem Namen `<testcase>` gefunden wird, wird überprüft, ob dieses Element die Attribute `name` und `classname` enthält. Ist dies der Fall, so wird das Element zu der Menge an bereits durchgeführten Testcases hinzugefügt.

Durch diese Art der Implementierung können zwar auch Testausgaben in JUnit XML geparkt werden, welche die Attribute `name` und/oder `classname` nicht benutzen (siehe Codeblock 3.4), jedoch wird durch das Fehlen von Attributen jeder Testcase als nicht ausgeführt betrachtet, was die Anwendung des Verfahrens hinfällig macht, da kein Testcase als durchgeführt betrachtet wird und in jedem Durchlauf des Verfahrens jeder Testcase ausgeführt wird.

Für die genaue Implementierung des Parsers im Code, siehe das in Kapitel 7 verlinkte Repository für `testAuditor`.

4.1.4 `testExecution`

Die Komponente `testExecution` ist eng mit der Komponente `testDiscovery` verwoben, denn die `testExecution` erwartet eine Menge von Namen, über welche sie gezielt die beschriebene Menge an Testcases an das Testframework übergibt, welches diese Menge dann ausführt. Die genaue Struktur dieser Namen wird durch das jeweilige Testframework vorgegeben, und wird von der `testDiscovery`

TODO

Ebenso wie die `testDiscovery` ist die `testExecution` hochgradig vom verwendeten Testframework abhängig, da sie direkt mit dem jeweiligen Testframework kommuniziert.

4.1.5 `controller`

Als `controller` wird jene zentrale Komponente beschrieben, welche als einzige Komponente direkt mit dem `user` interagiert, und sich mit der Ausführung der einzelnen Komponenten des Verfahrens beschäftigt, sowie I/O Prozesse der

Komponenten und Fehlerereignisse verwaltet.

Typischerweise erfüllt eine CI-Pipeline die Rolle des **controllers**, allerdings kann der **controller** auch auf andere Weisen implementiert werden.

4.1.6 user

Die Bezeichnung der Komponente **user** ist bewusst breit interpretierbar gehalten. Sie beschreibt jene Instanz, die mit dem **controller** interagiert, so den Ablauf des Verfahrens anstößt und/oder die Ausgabe des Verfahrens konsumiert. Dabei kann es sich um einen menschlichen Benutzer, oder um automatisierte Prozesse handeln.

4.2 Anforderungen an ein Testframework

Das beschriebene Verfahren kann mit jedem Testframework angewandt werden, welches diese Vorraussetzungen erfüllt, die sich unter Anderem auf Kapitel 3.3 ergeben:

- Eine beliebig kleine Untermenge aller Testcases kann gezielt ausgeführt werden
- JUnit XML wird als Ausgabeformat für Testdurchläufe verwendet
- Ein Testcase kann eindeutig über die Kombination der Attribute **name** und **classname** identifiziert werden

4.3 Anforderungen an ein Projekt

Nachdem die Anforderungen und Funktionsweisen der beteiligten Komponenten und Dateien ausführlich diskutiert wurden, können daraus nun die Anforderungen an ein Projekt abgeleitet werden, die vorausgesetzt werden, um das Verfahren implementieren zu können:

- Determinismus ist gegeben - Jeder Testdurchlauf erzeugt bei gleichem

Input und gleicher Konfiguration der Testcases das selbe Ergebnis, unabhängig von der Entwicklungsumgebung

- Es wird eine Komponente zur Verfügung gestellt, welche die in Kapitel 4.1.1 beschriebene Funktionalität bereitstellt
- Das verwendete Testframework erfüllt alle in Kapitel 4.2 dargestellten Anforderungen

4.4 Exemplarischer Ablauf

5 Evaluation der Benchmark

In diesem Kapitel werden die Ergebnisse der begleitenden Benchmark (siehe Kapitel 7) präsentiert und erörtert.

5.1 Aufbau der Benchmark

Die hier benutzte Benchmark ist in python für das Framework pytest geschrieben, und untersucht sechs verschiedene Basis-Konfigurationen. Diese Konfigurationen unterscheiden sich in der Gesamtzahl der Testcases, die ausgeführt werden, sowie der Anzahl an Iterationen, die je Testcase ausgeführt werden, was die Ausführungsdauer eines Testcase bestimmt. Jede Konfiguration wird mit sechs verschieden gefüllten **archives** getestet.

Dabei werden die sogenannten parametrisierten Tests von pytest genutzt, dargestellt in Codeblock 5.1:

```
@pytest.mark.parametrize("case", range(10))
def test_simple(case):
    index = 0
    for i in range(100):
        index += 1
    assert index == 100
```

Codeblock 5.1: Inhalt von `test_simple.py` aus der Konfiguration `"bench_fast_few_00"`

`range(x)` in der obersten Zeile bestimmt, wie viele Kopien des Testcase mit durchnummerierten Namen beim Ausführen des Testcase erstellt werden.

`range(y)` in der for-Schleife in der vierten Zeile bestimmt die Anzahl der Iterationen, die im Testcase ausgeführt werden, bevor er terminiert.

Das in Codeblock 5.1 demonstrierte Beispiel erzeugt also 10 Testcases, welche jeweils 100 Iterationen ausführen. Die Konfigurationen werden in 3 Geschwindigkeiten unterteilt: *fast*, *medium* und *slow*. Für jede dieser Geschwindigkeiten existieren zwei Untergruppen: *few* bezeichnet eine Konfiguration mit wenigen

Testcases, *many* eine mit vielen. Dabei wird die Gesamtmenge der Iterationen zur Erhaltung der Vergleichbarkeit innerhalb einer Geschwindigkeitsklasse gleich gehalten. Ein Beispiel: Während die Konfiguration "fast_few" 10 Testcases mit je 100 Iterationen enthält, enthält die Konfiguration "fast_many" 100 Testcases mit je 10 Iterationen. Somit werden in beiden Konfigurationen $10 \times 100 = 1000$ Iterationen ausgeführt, jedoch in unterschiedlichen Strukturen.

Jede Konfiguration enthält außerdem die Datei out.xml im Ordner archive. Diese Datei enthält das Ergebnis von vorherigen Testläufen, die vom Verfahren genutzt werden können, und erfüllt damit die Rolle der **archive**-Komponente (siehe Kapitel 4.1.2). Dabei existieren für jede der sechs bisher vorgestellte Kombination aus Testcaseanzahl und Iterationsanzahl sechs Konfigurationen: Eine Konfiguration enthält ein leeres **archive**, folgend gibt es Konfigurationen mit **archives**, die jeweils 20, 40, 60, 80 und abschließend 100 Prozent der Testcases in der Konfiguration abdecken.

Durch die drei Geschwindigkeitsklassen, die beiden unterschiedlichen Mengen von Testcases, und den unterschiedliche gefüllten **archives** beträgt die Gesamtzahl der in der Benchmark überprüften Konfigurationen 36.

Die Komponenten **testDiscovery** und **testExecution** werden durch zwei gleichnamige scripte zur Verfügung gestellt. Das script benchmark.sh, welches den Ablauf der Benchmark steuert, übernimmt die Rolle des **controller**.

5.1.1 Spezifikationen des ausführenden Systems

Der Computer, auf dem die Benchmark ausgeführt wurde, weist folgende Hardware-Spezifikationen auf:

- Prozessor: AMD Ryzen 5 7600, 3801 MHz, 6 Kerne, 12 logische Prozessoren
- RAM: 32,0 GB DDR5-6000
- Speicher: 1TB NVMe SSD

Die Benchmark wurde unter wsl ausgeführt. Nähere Informationen zur Softwareumgebung sind aus der Datei `flake.lock` im Repository der Benchmark (siehe Kapitel 7) zu entnehmen.

5.2 Datenerhebung der Benchmark

Es gilt zu beachten, dass die Benchmark die Dauer des gesamten Prozesses der Testerstellung misst. Für das in dieser Bachelorarbeit vorgestellte Verfahren bedeutet dies, dass die Ausführung der `testDiscovery`, der Ablauf von `testAuditor` sowie der Ablauf der `testExecution` gemessen werden. Die `testDiscovery` sowie die `testExecution` sorgen dabei für einen Testframeworkabhängigen Overhead.

Um einen direkten Vergleich der Ergebnisse zu ermöglichen, werden für jeden Durchlauf einer Konfiguration zwei individuelle Durchläufe gestartet:

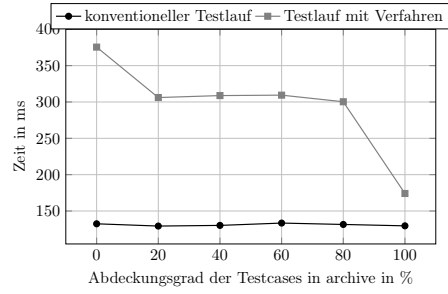
Der erste Lauf besteht auf der konventionellen Ausführung von Tests, bei der alle vorhandenen Testcases neu ausgeführt werden. Der zweite Lauf benutzt das in dieser Bachelorarbeit entwickelte Verfahren, um auf Grundlage der im `archive` liegenden Testausgaben die Mehrfache Ausführung von Testcases zu vermeiden.

Um die Auswirkungen von äußeren Einflüssen aus dem System auf den Ablauf der Benchmark bei der Datenerhebung auf die Ergebnisse zu verringern, wird die gesamte Benchmark zehnmal vollständig hintereinander ausgeführt. Das durchschnittliche Ergebnis ist dann die Grundlage für Kapitel 5.3.

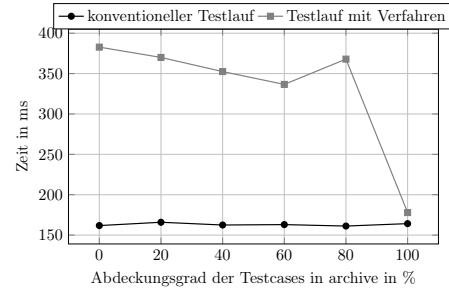
5.3 Ergebnisse der Benchmark

T bezeichnet die Anzahl an Testcases in der Konfiguration.

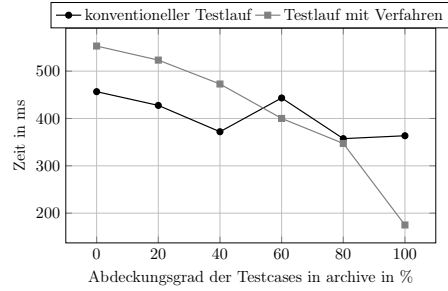
I bezeichnet die Anzahl an Iterationen pro Testcase.



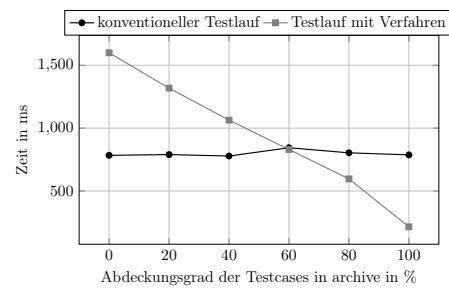
(1) wenige kurze Testcases: $T=10, I=100$



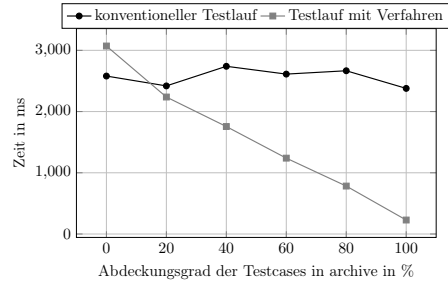
(2) viele kurze Testcases: $T=100, I=10$



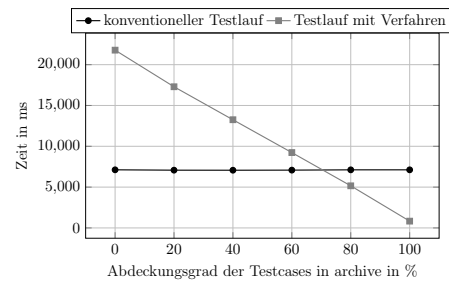
(3) wenige mittellange Testcases: $T=10, I=1.000.000$



(4) viele mittellange Testcases: $T=1.000, I=10.000$

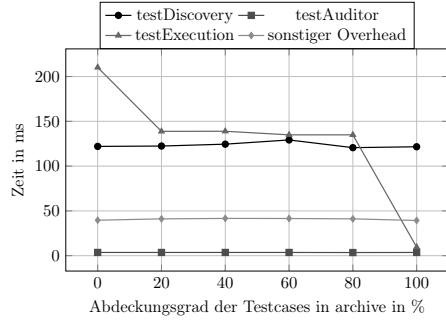


(5) wenige langsame Testcases: $T=10, I=10.000.000$

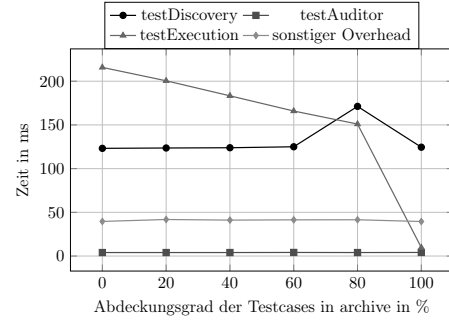


(6) viele langsame Testcases: $T=10.000, I=10.000$

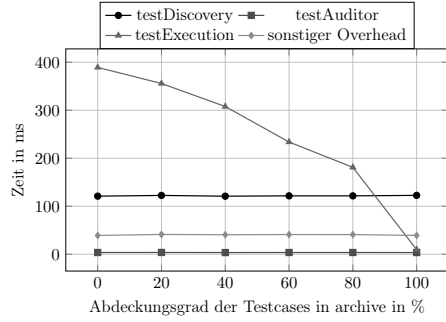
Abbildung 2: Ergebnisse der Benchmark. Sämtliche dieser Übersicht zugrunde liegenden Daten können in dem Repository der Benchmark (siehe Kapitel 7) eingesehen und überprüft werden.



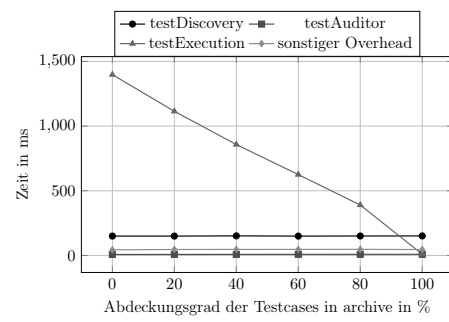
(a) wenige kurze Testcases: $T=10$, $I=100$



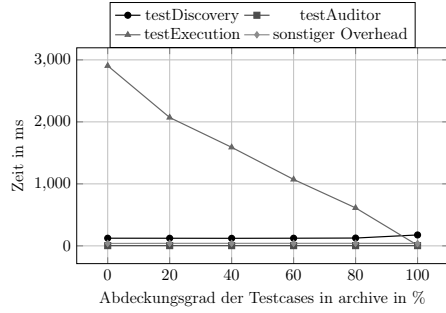
(b) viele kurze Testcases: $T=100$, $I=10$



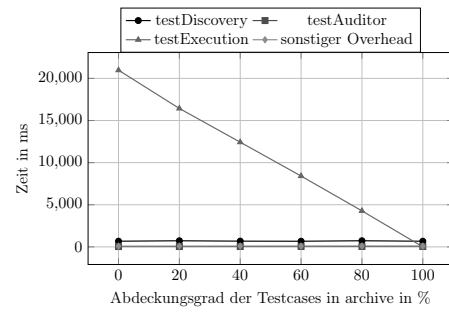
(c) wenige mittellange Testcases: $T=10$, $I=1.000.000$



(d) viele mittellange Testcases: $T=1.000$, $I=10.000$



(e) wenige langsame Testcases: $T=10$, $I=10.000.000$



(f) viele langsame Testcases: $T=10.000$, $I=10.000$

Abbildung 3: Detailansicht zu den Laufzeiten einzelner Komponenten des Verfahrens. Sämtliche dieser Übersicht zugrunde liegenden Daten können in dem Repository der Benchmark (siehe Kapitel 7) eingesehen und überprüft werden.

5.4 Zusammenfassung der Ergebnisse

Aus Abbildung 2 ist ersichtlich, dass sich mit der Anwendung der Verfahrens durchaus eine Verringerung der Laufzeit eines Testdurchlaufs erreichen lässt. Durch die Verwendung der verschiedenen Komponenten des Verfahrens und der notwendigen Kommunikation dieser untereinander entsteht zwar ein gewisser Overhead im Vergleich zu einem konventionellen Testdurchlauf, dieser nimmt jedoch mit zunehmender Abdeckung von Testcases in **archive** nahezu konstant ab.

Jedoch decken die Graphen (2), (4) und (6) einen Makel auf: Bei einer hohen Anzahl an Testcases, die ausgeführt werden müssen, ist die Laufzeit des Verfahrens zum Teil deutlich höher als ein konventioneller Durchlauf. Abbildung 3 liefert die Erklärung zu diesem Verhalten: Während die Laufzeiten für die **testDiscovery**, den **testAuditor** und für sonstigen Overhead, der durch das ausführende Script entsteht, innerhalb einer Konfiguration der Benchmark nahezu konstant bleiben, ist die Laufzeit der **testExecution** stark von der vorhandenen Menge an Testcases abhängig. Vergleicht man einzelne Graphen von der gleichen Konfiguration aus Abbildung 2 und Abbildung 3 miteinander, so ist ersichtlich, dass die **testExecution** bei auch gleicher Menge an Testcases deutlich langsamer ist als ein konventioneller Testlauf. Dies lässt sich durch das verwendete Testframework und die Funktionsweise der **testExecution** erklären:

Die **testExecution** bekommt von **testAuditor** eine Menge an Namen von Testcases, die ausgeführt werden sollen. In dieser Benchmark wird diese Menge von der **testExecution** an **pytest** übergeben, was bei einer hohen Anzahl von Testcases eine sehr teure Operation werden kann. Außerdem ist **pytest** nicht effizient auf diese Art von Benutzung ausgelegt, sodass die Laufzeit bei dieser Art von Aufruf mit vielen Testcases deutlich schlechter als ein kompletter Neudurchlauf ist. Der Grad der Auswirkung dieser Implementationsweise ist also auch abhängig vom verwendeten Testframework.

Die beste Performance zeigt das Verfahren jedoch in Fällen wie in Graph (5)

demonstriert:

Bei einer geringen Anzahl fällt die langsame Implementation der `testExecution` weniger ins Gewicht, und bei einer langen Laufzeit der einzelnen Testcases kann durch das Vermeiden des Ausführens von Testcases, für die bereits ein Ergebnis vorliegt, eine signifikante Verringerung der Gesamtlaufzeit des Testdurchlaufs erzielt werden. So performt das Verfahren in (5) ab einer Abdeckung der Testcases von etwa 20% bereits besser als der konventionelle Durchlauf.

Je mehr Testcases bereits ausgeführt wurden, desto bessere Laufzeiten erzielt das Verfahren. Selbst bei Konfigurationen mit vielen Testcases performt das Verfahren ab einer Abdeckung der Testcases durch `archive` von etwa 80% besser als ein konventioneller Durchlauf. Eine Ausnahme bilden Konfigurationen mit Testcases mit sehr kurzer Laufzeit: Hier ist der Laufzeitgewinn pro gespartem Testcase gering, und der Overhead überwiegt die Laufzeit eines konventionellen Durchlaufs, wie in (1) und (2) ersichtlich.

6 Fazit

6.1 Erfüllung der Anforderungen

Im Folgenden soll nun untersucht werden, ob die in Kapitel 2.1 gestellten Anforderungen unter Berücksichtigung der Ergebnisse von Kapitel 5.3 durch das Verfahren erfüllt werden konnten.

6.2 Möglichkeiten zur Verbesserung

TODO: Überarbeiten.

In seiner aktuellen Form kann das Tool nicht entscheiden, ob die XML-Ausgabe der Tests auch tatsächlich aus dem aktuellen Stand des Codes entstammt, oder ob im Nachhinein Änderungen an dem zu testenden Code oder den Tests selber vorgenommen wurden, was die Aussagekraft des Tools stark beeinträchtigt.

Ein anderes Problem ist, dass das Tool alle Tests, die nicht an den letzten Schwung gitnotes anhängt sind, als nicht abgelaufen betrachtet, auch wenn einige Testfälle in einer vorherigen Iteration ausgeführt sein könnten, und die Änderungen, die seither im Code gemacht wurden diese Tests überhaupt nicht betreffen.

Eine Möglichkeit, dieses Problem zu beheben, wäre die Integration eines neuen Tools, welches eine Seletive Regression Testing / Regression Test Selection implementiert, und so dem Tool rückmelden kann, welche Tests tatschlich neu ausgeführt werden müssen, und bei welchen der vorherige Stand noch gültig ist. Zusätzliche Unterstützung von TAP wäre realistisch, JSON formate eher unrealistisch. Robuster machen für name+classname kombis -> Ekstazi <https://github.com/gliga/ekstazi>

Literaturverzeichnis

- [1] Mauro Pezzè und Michal Young. *Software Testing and Analysis: Process, Principles, and Techniques*. John Wiley & Sons, 2008. ISBN: 978-0-471-45593-6.
- [2] Dietmar Winkler, Reinhard Hametner und Stefan Biffl. “Automation Component Aspects for Efficient Unit Testing”. In: *2009 IEEE Conference on Emerging Technologies & Factory Automation*. IEEE, 2009. DOI: 10.1109/ETFA.2009.5347022.
- [3] Isaac Z. Schlueter. *TAP14 - The Test Anything Protocol v14*. Archivierte Version vom 10.07.2026, verfügbar über die Wayback Machine: <https://web.archive.org/web/20260710220327/https://testanything.org/tap-version-14-specification.html>. 2022. URL: <https://testanything.org/tap-version-14-specification.html> (besucht am 10.07.2026).
- [4] Marc Philipp. *STF Milestone 2: Open Test Reporting*. Archivierte Version vom 11.07.2026: <https://web.archive.org/web/20260710233305/https://marcphilipp.de/blog/2025/03/21/stf-milestone-2-open-test-reporting/>. 21. März 2025. URL: <https://marcphilipp.de/blog/2025/03/21/stf-milestone-2-open-test-reporting/> (besucht am 10.07.2026).

7 Links

Latex-Code der vorliegenden Bachelorarbeit:

https://github.com/Zip-creations/BA_latex

Evaluation der Testausgabeformate ausgewählter Testframeworks:

https://github.com/Zip-creations/evaluate_testformats

Quellcode von testAuditor:

<https://github.com/Zip-creations/testAuditor>

**Exemplarischer Aufbau des Verfahrens in einem Python-projekt mit
pytest:**

https://github.com/Zip-creations/BA_showcase

Benchmark mit allen .log-Dateien:

https://github.com/Zip-creations/BA_benchmark